
PATTERNS EDITION / 1.0

LeetCode Bible

Learn to derive algorithms,
explain why they work,
and solve unfamiliar problems.

01 CONTRACT

02 BASELINE

03 OBSERVATION

04 INVARIANT

05 IMPLEMENTATION

About this edition

Patterns edition

Learning to derive, explain, and implement algorithms in Rust

Edition 1.0 | 6 October 2026

This book is a working guide to solving unfamiliar algorithm problems independently. It explains the observations that make techniques useful, the conditions that make them correct, and the changes that make them fail. Read it with a notebook and an editor. The exercises are part of the book.

The patterns edition covers the main interview algorithm families and selected practice problems. It does not contain every LeetCode problem or reproduce the platform's problem statements. Problem links lead to the original statements. The companion repository provides implementation references; consult them after your attempt.

The book's Rust functions are standalone teaching implementations. Their `Option` return values and slice-based interfaces may differ from LeetCode's required submission signatures. Adapt the interface without changing the algorithm's contract.

The explanations, examples, practice sequence, and exercises here were written for this book. Research citations distinguish evidence about learning from practical recommendations. No number of completed problems guarantees mastery.

Book text: CC BY-SA 4.0. Original code examples and build tools: MIT. See the source package for license texts, attribution, and contribution instructions. Referenced third-party materials retain their own licenses. This is an independent educational project, unaffiliated with LeetCode.

Contents

1 How to use this book	5
2 Derive and justify a solution	6
3 Reasoning moves that unlock problems	7
4 A map of useful observations	10
5 Hashing and sufficient summaries	11
6 Prefix sums and algebraic transformations	12
7 Two pointers and safe elimination	13
8 Sliding windows and monotone validity	14
9 Binary search over a boundary	15
10 Stacks and monotonic deques	16
11 Heaps and the candidate frontier	17
12 Sorting intervals and greedy choices	18
13 Linked lists and recursive trees	19
14 Backtracking and the search tree	20
15 Graph modeling and traversal	21
16 Dependencies and merging components	22
17 Weighted shortest paths	23
18 Dynamic programming from sufficient state	24
19 Dynamic programming families	25
20 Tries and string structure	26
21 Bits mathematics and small state spaces	27
22 Range queries under updates	28

23 Data structure design and composed invariants	29
24 Rust as an implementation tool	30
25 Similar looking problems with different rules	31
26 A curriculum built around dependencies	32
27 Transfer exercises	33
28 Hints and transfer exercise answers	34
29 Review pages and contribution method	35
30 Advanced families and when they become useful	36
31 Sources and further reading	38
32 Selected problem index	40

1 How to use this book

Your target is a repeatable ability: read a new problem, construct a correct approach, improve it when necessary, explain why it works, and implement it without someone supplying the key decisions.

Work in two modes. In learning mode, read a chapter, study a worked example, and solve a related problem with help when needed. In testing mode, hide tags and notes, mix previously studied topics, and attempt an unfamiliar problem. Record these accomplishments separately. A solution you reconstructed after a hint is progress, but it is different evidence from an independent solution.

For each learning problem, use this sequence:

1. Restate the input, output, constraints, and ambiguous cases.
2. Work through a small example by hand.
3. Describe a straightforward correct algorithm and its cost.
4. Identify exactly which work is repeated or unnecessary.
5. Propose an observation that removes that work.
6. State what your algorithm maintains and why each update is safe.
7. Implement, trace, and test it.
8. Close any explanation and reconstruct the missing reasoning.
9. Return after a delay, then attempt a different problem using the idea.

A practical starting point is 20-30 minutes of serious effort before a hint, adjusted to your background and the problem. Keep thinking while you are generating useful alternatives. Study a worked example when a technique is entirely new. Exact time limits and review intervals are coaching choices, not experimentally established LeetCode optima. Experienced competitors differ on editorial timing [S4](#).

On review, write the argument before looking at code. A day or two, a week, and several weeks are reasonable initial review intervals; shorten or lengthen them based on your recall. Retrieval experiments support delayed recall practice, though the classic study used prose passages [S6](#).

For a 60-minute session, try 10 minutes of retrieval, 30 minutes of a new attempt, 15 minutes of targeted explanation or reconstruction, and 5 minutes recording your missing insight. Adjust the balance as needed. Once several techniques are available, include mixed sessions. Interleaving has experimental support in mathematics; applying it to algorithm selection is a reasoned extrapolation [S7](#).

A useful progress record: problem; independent or assisted; first incorrect assumption; missing observation; invariant; implementation error; next variation. Record evidence, such as "explained why the left endpoint never returns," instead of a vague confidence rating.

2 Derive and justify a solution

An optimization should answer a specific complaint about a correct baseline. "Too slow" is insufficient. Does the baseline repeatedly search old elements? Recompute an overlapping sum? Explore different histories that reach the same future? Keep candidates that are permanently worse than another candidate?

Consider searching for two values that total a target. Enumerating every pair costs quadratic time. For each value x , the actual question is whether $\text{target} - x$ has appeared earlier. A lookup structure removes the repeated scan. The arithmetic creates the lookup key; the hash map follows from the required operation.

Now consider maximizing a contiguous sum. A negative running prefix can only make a future extension worse than starting after it. Discarding that prefix is justified by comparing both candidates with the same possible future suffix. This is a dominance argument: one candidate is no better under any relevant future continuation.

A correctness argument has three parts for an iterative invariant:

1. **Initialization:** the claimed fact is true before the first iteration.
2. **Maintenance:** each update preserves it.
3. **Completion:** the fact, together with termination, implies the requested answer.

"The window is valid" may be necessary, but it does not prove optimality. You must also explain why discarded windows cannot improve the answer, and why every relevant candidate is considered or safely represented.

For recursion, specify the contract of one call: what its inputs mean and exactly what it returns. Assume smaller calls satisfy that contract and prove how their results produce the current answer. For greedy choices, give an exchange or dominance argument. For dynamic programming, establish sufficient state, complete transitions, and an evaluation order with dependencies already available.

Complexity follows the actual operations. A loop inside a loop is not automatically quadratic: if a pointer advances at most n times across the entire execution, those advances total $O(n)$. Conversely, slicing, cloning, hashing a long string, or copying a path can add work that the visible loop structure hides.

Growth	What to notice
$O(n)$	Every element contributes bounded work overall.
$O(n \log n)$	Sorting or logarithmic updates for each item.
$O(n \text{ squared})$	Every pair, or n states with n transitions each.
$O(2 \text{ to the } n)$	Independent include/exclude decisions; often small n only.
$O(n \text{ factorial})$	Enumerating orders of distinct items.
$O(V + E)$	Visit graph vertices and adjacency entries a bounded number of times.

Constraints filter candidates; they do not determine an algorithm uniquely. There is no universal operation-count threshold across languages, machines, and judges. Include output size and recursion stack in your analysis.

3 Reasoning moves that unlock problems

The most useful mental shortcuts reduce a vague search for a technique to a concrete question. Each move below proposes something to investigate. None excuses checking assumptions or proving correctness.

Ask what information the future needs

Two histories can share a state only if the remaining legal choices and their consequences are equivalent for your objective. A path through a grid may need only its current cell for reachability, but needs a key mask when doors depend on collected keys. A shopping problem may need remaining budget and position; the exact earlier purchase order may be irrelevant.

Try this sentence: "If I tell you these variables, can you solve the remainder without knowing how I got here?" If the answer is no, find the missing variable. If yes, look for repeated states and memoization. State compression is justified forgetting.

Write the query hidden inside the loop

Before choosing a collection, name the operation in plain language: "Have I seen this complement?", "What is the earliest index with this prefix?", "What is the smallest available finishing time?", or "How many earlier values are below this threshold?"

The answers suggest a hash map, a map retaining earliest positions, a heap, or an ordered counting structure. This also distinguishes superficially similar problems: membership, frequency, rank, and minimum are different queries.

Change the representation before changing the algorithm

A representation can expose structure that was hidden in the story. Turn matching zeros and ones into a sum by mapping zero to -1 and one to +1; a balanced range then has sum zero. Turn a sequence of allowed moves into graph edges. Turn time intervals into start and end events.

For [LC 525 Contiguous Array](#), repeated transformed prefix sums identify balanced ranges, and keeping the earliest boundary maximizes length. The observation is an algebraic encoding followed by prefix lookup. "Use a hash map" alone misses the discovery.

Fix one dimension and solve the remainder

Three Sum fixes one value and solves a pair problem on the suffix. Counting triangles in sorted positive lengths can fix the largest side and reason about pairs. Fixing a boundary, center, last value, or number of chosen items can reduce a tangled problem to a simpler one.

Account for the outer enumeration. An $O(n)$ inner method repeated for n fixed values costs $O(n^2)$. Avoid fixing a dimension that destroys the structure needed by the remaining algorithm.

Ask for a decision before an optimum

Finding the best value directly can be difficult while checking a proposed value is easy. Replace "What is the smallest capacity?" with "Can this capacity work?" Then prove how feasibility changes as capacity increases.

The essential work is often the decision procedure: a greedy packing argument, a graph traversal under a threshold, or a DP. Binary search only accelerates repeated decisions. If checking is expensive or nonmonotone, the transformation may not help.

Compare candidates under the same future

To justify discarding A in favor of B, attach the same arbitrary future continuation to both. If B is always at least as feasible and at least as good, A is dominated.

This explains a smaller tail for an increasing subsequence of the same length, a newer larger value in a maximum-window deque, and a later no-larger prefix in the shortest-sum deque. The dimensions differ, but the proof structure is shared. If candidates trade off cost and remaining resources, neither may dominate; retain a richer frontier or enlarge the state.

Reverse time or choose the last action

Deletions can be hard for a data structure that supports additions. If all operations are known beforehand, reversing their order may turn deletions into insertions. This does not solve online deletions; the availability of the whole history is the enabling condition.

Choosing the last action is another reversal. In interval problems such as [LC 312 Burst Balloons](#), selecting what disappears last inside an interval fixes its surviving outside neighbors. Earlier actions can then be assigned to independent left and right subproblems. Ask whether choosing the last event creates stable boundaries.

Count each element's contribution

Instead of generating every object and computing its value, ask how many objects contain a particular contribution. Summing all subarray sums can be done by counting how many choices of left and right endpoints include each index i : $(i + 1)$ times $(n - i)$.

For [LC 907 Sum of Subarray Minimums](#), count the ranges for which each element is the designated minimum. Nearest smaller boundaries determine the choices. Equal values require asymmetric strictness on the two boundaries, or another consistent tie rule, so each range is counted once. The contribution trick moves difficulty into assigning ownership correctly.

Count a complement or subtract two easier counts

Sometimes the desired set is awkward but the universe and unwanted set are easy to count. Another useful identity is $\text{exactly}(k) = \text{at_most}(k) - \text{at_most}(k - 1)$, when at_most counts nested sets under the same statistic.

For ranges with exactly k distinct values, an at-most- k frequency window can count all valid starts for each endpoint. This is a counting argument beyond merely maintaining one valid window. The identity is algebraically valid, but the efficient at-most procedure still needs its own proof. Guard the $k = 0$ and negative-threshold cases.

Process events in an order that makes information final

Sorting can convert a changing eligibility rule into one-way progress. For an offline query asking for the smallest interval containing a point, sort query points, insert intervals whose start has become eligible, and remove intervals whose end is already behind the point. A heap can choose the shortest remaining interval. Restore original query order in the output.

This composition appears in [LC 1851 Minimum Interval to Include Each Query](#). The question to ask is: "Can I reorder the work so candidates only enter or leave once?" Reordering requires an offline or otherwise order-independent contract.

Split an exponential search into two halves

If enumerating all subsets is too large but n is still moderate, enumerate sums for each half separately. Sort one half's sums and search for compatible sums from the other. This changes the main enumeration from 2 to the n toward 2 to the $n/2$, at a substantial memory cost.

Extra constraints must survive the split. When exactly half the elements must be chosen, bucket half-sums by the number selected and combine compatible bucket sizes. [LC 1755 Closest Subsequence Sum](#) and [LC 2035 Partition Array Into Two Arrays to Minimize Sum Difference](#) illustrate the distinction. A target-sum DP may instead be better when the numeric range is small.

Use a lower bound and match it with a construction

To prove an optimum, first show that no answer can beat a certain bound. Then construct an answer that reaches it. Capacity, parity, maximum frequency, or a bottleneck can supply such a bound.

Be careful: necessary conditions need not be sufficient. A rearrangement may satisfy a total-count bound yet fail a local constraint. Until you construct a valid object or prove existence, the bound is only a bound. Scheduling formulas deserve the same scrutiny as algorithms.

Attack your own idea with a tiny counterexample

Before coding a greedy rule, try three or four items with conflicting incentives. To challenge a sum window, include a negative value. To challenge a count, include duplicates and zeros. To challenge a graph state, revisit a vertex with different resources. To challenge a boundary, test empty input, one item, all equal values, and an absent answer when permitted.

Keep shrinking a failing input while preserving the failure. A six-element counterexample may expose the missing assumption much more clearly than a hundred-element random case. Counterexample construction is a problem-solving skill: it tells you what the next algorithm must remember or prove.

Distinguish recognition from derivation

It is reasonable to recognize a familiar structure immediately. You do not need to rediscover established algorithms in every interview. The useful check is whether you can explain why the structure applies and adapt it when a condition changes.

If a solution feels like a trick, locate the first unsupported jump. Was a predicate's monotonicity assumed? Was information discarded without a dominance argument? Was a DP state chosen without checking future equivalence? Repair that jump. The rest of the code often becomes routine once the decision is justified.

4 A map of useful observations

Use this table to generate hypotheses. Verify the condition in the middle column before selecting a technique.

Observation	Condition to establish	Candidate technique
Earlier information answers a repeated query	A compact lookup key captures what matters	Hash map or set
Many ranges reuse the same accumulated work	The aggregate can be recovered from prefixes	Prefix accumulation
A comparison eliminates a whole region	Eliminated candidates cannot satisfy or improve the objective	Two pointers or binary search
Nearby windows share most elements	Updates are cheap; endpoint movement is justified	Sliding window
Only a few best candidates matter	Discarded candidates cannot re-enter under future operations	Heap or dominance frontier
Some candidates wait for a resolving event	Candidates become resolved or dominated in an ordered way	Monotonic stack or deque
Different histories have identical futures	A bounded state describes every relevant future choice	Memoization or DP
Choices form paths between states	States and legal transitions can be defined precisely	Graph search
Work must precede other work	Dependencies are directed and acyclic	Topological processing
Groups merge repeatedly	Connectivity only grows, or time can be reversed	Union-find
A local choice preserves an optimal completion	An exchange or dominance proof exists	Greedy
Many queries overlap while values change	An aggregate combines associatively	Fenwick or segment tree

These ideas compose. A scheduler might sort jobs by arrival, sweep time, keep available jobs in a heap, and choose one by a greedy rule. Each component has its own purpose and correctness condition. Calling the entire solution a "heap problem" loses that structure.

Patterns also overlap. A memoized DFS can be dynamic programming. A monotonic deque can maintain a window. A shortest-path problem can be dynamic programming on an acyclic state graph. Learn the behavior and proof first; labels help organize it afterward.

5 Hashing and sufficient summaries

Hashing is useful when you repeatedly ask about previously processed information: membership, frequency, an index, or the best value associated with a key. Derive the key from the question. Two Sum uses a complement. Anagrams use a representation of character multiplicities. Prefix-sum counting uses a previous accumulated value.

For Two Sum, process left to right. Before examining index i , the map contains a usable index for each value before i . Query the complement before inserting the current value so an element cannot pair with itself. Duplicate values are allowed: target 6 and values [3, 3] require two positions.

```
pub fn two_sum(nums: &[i32], target: i32) -> Option<(usize, usize)> {
    use std::collections::HashMap;
    let mut seen = HashMap::<i64, usize>::new();
    for (i, &x) in nums.iter().enumerate() {
        let x = i64::from(x);
        if let Some(&j) = seen.get(&(i64::from(target) - x)) {
            return Some((j, i));
        }
        seen.insert(x, i);
    }
    None
}
```

Under expected constant-time integer hashing, time and auxiliary space are $O(n)$. Widening arithmetic before subtraction avoids $i32$ overflow. The map may replace an older index for a value because any earlier occurrence is sufficient for this contract. For the longest matching range, that replacement could be wrong: the earliest index may be essential.

What changes the method: requesting all pairs changes output size and duplicate handling; requesting the count requires frequencies; preserving order can make sorting unsuitable. If the key is a string of length m , building and hashing it generally costs $O(m)$.

For grouping anagrams over lowercase English letters, a 26-element count vector is a canonical key: order is irrelevant but multiplicity matters. A set of letters would confuse "abb" with "ab". For a wider alphabet, define the representation explicitly.

Practice: [LC 1 Two Sum](#), [LC 242 Valid Anagram](#), [LC 49 Group Anagrams](#), [LC 128 Longest Consecutive Sequence](#). In the last problem, begin counting only at values with no predecessor; otherwise repeated scans can become quadratic.

Check your understanding: What information may a map safely forget? Why does checking before insertion matter? What changes when the answer asks for the earliest, latest, longest, or total number?

6 Prefix sums and algebraic transformations

Define $P[0] = 0$ and $P[i + 1] = P[i] + a[i]$. Then the sum of the half-open range $[l, r)$ is $P[r] - P[l]$. The leading zero represents the empty prefix and makes ranges starting at zero ordinary cases.

For many static range queries, this changes $O(\text{length})$ work per query to $O(1)$, after $O(n)$ preprocessing and space. For finding subarrays with sum k , rearrange $P[r] - P[l] = k$ into $P[l] = P[r] - k$. The problem becomes counting previous prefixes with a particular value.

```
pub fn count_sum(nums: &[i32], k: i64) -> i64 {
    use std::collections::HashMap;
    let mut counts = HashMap::from([(0_i64, 1_i64)]);
    let (mut prefix, mut answer) = (0_i64, 0_i64);
    for &x in nums {
        prefix += i64::from(x);
        if let Some(needed) = prefix.checked_sub(k) {
            answer += counts.get(&needed).copied().unwrap_or(0);
        }
        *counts.entry(prefix).or_insert(0) += 1;
    }
    answer
}
```

Before each query, counts describes prefix boundaries strictly before the current boundary. Querying before insertion excludes an empty subarray when k is zero. For $[1, -1, 1]$ and $k = 1$, the answer is 3. Keeping only a set would lose the multiplicity of repeated prefix values.

This method allows negative elements; no monotone sum is required. Expected time is $O(n)$, space $O(n)$. The code assumes accumulated sums and the count fit $i64$.

Prefix accumulation is broader than addition. XOR ranges use $P[r] \text{ XOR } P[l]$. A prefix minimum cannot generally recover an arbitrary range minimum by subtraction because the operation has no suitable inverse. Product queries need care around zeros, overflow, and division.

A difference array reverses the perspective. Adding x to every element of $[l, r)$ is represented by adding x at boundary l and subtracting x at r . A final prefix pass reconstructs all point values. This is useful when updates can be batched before the answers are required.

Practice: [LC 303 Range Sum Query Immutable](#), [LC 560 Subarray Sum Equals K](#), [LC 974 Subarray Sums Divisible by K](#), [LC 1109 Corporate Flight Bookings](#), [LC 238 Product of Array Except Self](#). For divisibility, normalize negative remainders, and require a nonzero modulus.

Check your understanding: Why does the map start with one empty prefix? Which information must be stored to count ranges versus find the longest one? Why do point updates make a plain prefix array expensive to maintain?

7 Two pointers and safe elimination

Two pointers become an algorithm when you can justify moving one without reconsidering the discarded candidates. Their mere presence in code proves nothing about correctness or complexity.

In a sorted array, suppose $a[\text{left}] + a[\text{right}]$ is below the target. Every sum using this left value and a remaining index at or before right is also too small. Therefore left can advance. If the sum is too large, the right value cannot pair with any remaining index at or after left, so right can retreat. Each move eliminates a whole row or column of candidate pairs. Total movement is $O(n)$.

Sorting an unsorted input first costs $O(n \log n)$ and may require retaining original indices. If the requested result concerns original adjacency, sorting changes the problem and is not automatically allowed.

For removing duplicates from a sorted array, use a read boundary and a write boundary. The prefix before write contains exactly the distinct values encountered so far, in order. A new distinct value extends that prefix; a duplicate is skipped. The output region has a precise meaning even though the remaining array contains irrelevant data.

Three Sum fixes one value and uses the sorted-pair argument on the suffix. That composition gives $O(n^2)$ scanning after sorting. Skipping equal fixed values and equal pointer values controls duplicate triplets. The goal is unique value triplets; a different output contract would require different duplicate rules.

A different proof: in Container With Most Water, keeping the shorter boundary while narrowing the width cannot improve the area. Moving the taller boundary alone cannot escape the shorter height limit. This is a dominance argument, not the sorted-sum argument.

Fast and slow pointers use yet another fact. In a cycle of length c , a pointer moving twice as fast gains one position per step modulo c , so the pointers meet. Locating the cycle entrance requires an additional distance argument. Do not treat every two-pointer problem as one template.

Practice: [LC 167 Two Sum II](#), [LC 26 Remove Duplicates from Sorted Array](#), [LC 15 3Sum](#), [LC 11 Container With Most Water](#), [LC 141 Linked List Cycle](#), [LC 142 Linked List Cycle II](#).

Check your understanding: Name the entire set eliminated by one pointer move. Would that elimination remain valid without sortedness? Does the proof permit equality, and what happens to duplicates?

8 Sliding windows and monotone validity

A fixed-size window reuses information between adjacent ranges. Add the entering element and remove the leaving element. The sum of every k -element range can be computed in $O(n)$ time and $O(1)$ extra space, even if values are negative.

A variable-size window needs more: a reason that an endpoint can move permanently. For the shortest range reaching a positive target in an array of positive values, extending right increases the sum and advancing left decreases it. Whenever the sum is sufficient, record the length and shrink. Once a left boundary yields a valid window, later right endpoints cannot improve the shortest length for that same left boundary.

```
pub fn min_len_positive(nums: &[i32], target: i64) -> Option<usize> {
    assert!(target > 0 && nums.iter().all(|&x| x > 0));
    let (mut left, mut sum) = (0_usize, 0_i64);
    let mut best = None;
    for (right, &x) in nums.iter().enumerate() {
        sum += i64::from(x);
        while sum >= target {
            let len = right - left + 1;
            best = Some(best.map_or(len, |old: usize| old.min(len)));
            sum -= i64::from(nums[left]);
            left += 1;
        }
    }
    best
}
```

Each value enters and leaves at most once, so both loops together cost $O(n)$. The maintained sum equals the current range sum; the recorded best covers every discarded left boundary's shortest discovered valid range. Empty input produces `None`.

For `[2, 3, 1, 2, 4, 3]` and target 7, the first valid window has length 4. Later, shrinking `[2, 4, 3]` yields `[4, 3]`, improving the answer to 2. Draw the left and right indices as they advance; neither retreats.

The positive-sum rule fails on `[1, -1, 5]` with target 5. After finding the full range, removing 1 lowers the sum to 4, so that rule stops shrinking and misses `[5]`. Negative elements do not invalidate all sliding windows: they invalidate this monotonicity argument.

For longest substrings without repeated characters, maintain character frequencies and shrink until duplicates disappear. Adding characters cannot repair a duplicate, and removing enough characters can. For minimum covering windows, maintain required multiplicities; shrink while coverage remains sufficient, recording candidates before invalidating coverage.

Practice: [LC 643 Maximum Average Subarray I](#), [LC 209 Minimum Size Subarray Sum](#), [LC 3 Longest Substring Without Repeating Characters](#), [LC 424 Longest Repeating Character Replacement](#), [LC 76 Minimum Window Substring](#). Leave optimized stale-frequency tricks in LC 424 until you can prove the simpler version.

Check your understanding: Is this a fixed or variable window? What makes invalidity persist under expansion, or validity persist under expansion? When should the answer be recorded: before shrinking, during shrinking, or afterward?

9 Binary search over a boundary

Binary search needs an ordered search space and a safe elimination rule. For a first-true search, define a predicate that is false on an initial segment and true afterward. Keep a half-open candidate interval $[lo, hi)$. A true midpoint moves hi to mid ; a false midpoint moves lo past mid .

```
pub fn first_true<F>(n: usize, mut predicate: F) -> usize
where
    F: FnMut(usize) -> bool,
{
    let (mut lo, mut hi) = (0, n);
    while lo < hi {
        let mid = lo + (hi - lo) / 2;
        if predicate(mid) {
            hi = mid;
        } else {
            lo = mid + 1;
        }
    }
    lo
}
```

The result is n if no index satisfies the predicate. It never evaluates $\text{predicate}(n)$. All indices below lo are known false; all indices at or above hi within the domain are known true. Each iteration makes the interval smaller. The caller must supply a consistent monotone predicate.

A lower bound in sorted values uses the predicate $a[i] \geq \text{target}$. To search an answer, first turn optimization into a decision. For Koko Eating Bananas, ask whether a proposed speed finishes the work within the allowed hours. Increasing speed cannot make completion slower. Search the smallest feasible speed, after establishing a feasible upper bound.

The cost is $O(\log R \text{ times } C)$, where R is the discrete search range and C is the cost of checking one candidate. Calling it merely $O(\log n)$ hides a full scan if each check examines all elements. Use widened arithmetic for accumulated hours; compute ceiling division safely for the permitted numeric range.

Peak finding and rotated-array search have related elimination arguments, but their raw input values need not form a globally monotone predicate. Understand those arguments separately. A sorted input also admits techniques other than binary search.

Practice: [LC 704 Binary Search](#), [LC 35 Search Insert Position](#), [LC 34 Find First and Last Position](#), [LC 875 Koko Eating Bananas](#), [LC 1011 Capacity To Ship Packages Within D Days](#), [LC 33 Search in Rotated Sorted Array](#).

Check your understanding: State the exact yes/no question. Which implication proves monotonicity? What does an all-false domain return? Why can the loop not repeat the same interval? See [S9](#) for boundary invariants and answer search.

10 Stacks and monotonic deques

A stack models unfinished work in last-in, first-out order: nested delimiters, expression evaluation, recursive calls, or candidates waiting for a future event. For balanced parentheses, each closing symbol must match the most recent unmatched opening symbol. A count alone cannot distinguish correctly nested types.

In Daily Temperatures, keep indices whose next warmer day is unknown, with temperatures nonincreasing from bottom to top. A new strictly warmer temperature resolves indices at the top. The current day is their first warmer day because every earlier intervening day failed to resolve them. Each index is pushed once and popped once, giving $O(n)$ total time and $O(n)$ space.

Equal values matter. For "next strictly greater," equality does not resolve a waiting index. For "next greater or equal," it does. A monotonic stack's comparison operator encodes the problem's contract.

A deque adds expiration at the front. For a fixed-window maximum, keep indices in increasing time order and values decreasing from front to back. Remove expired indices at the front. Remove smaller or equal values at the back before adding a new value: the newer value is at least as large and expires later, so it dominates those older candidates. The front is the current maximum. Choosing to retain equals is also possible if expiration is handled correctly.

Shortest Subarray with Sum at Least K combines prefix sums and a deque. For prefix boundaries $j < i$, the range sum is $P[i] - P[j]$. Maintain increasing prefix values in the deque. If a later prefix is no larger than an earlier one, the later boundary gives at least as much future sum and a shorter future range, so the earlier boundary can be discarded. At the front, whenever a range reaches k , record it and remove that start: any later endpoint would make its length worse.

That proof explains both ends of the deque. It also explains why the positive-sum window from Chapter 8 cannot simply be reused with negative values.

Practice: [LC 20 Valid Parentheses](#), [LC 155 Min Stack](#), [LC 739 Daily Temperatures](#), [LC 496 Next Greater Element I](#), [LC 239 Sliding Window Maximum](#), [LC 862 Shortest Subarray with Sum at Least K](#), [LC 84 Largest Rectangle in Histogram](#).

Check your understanding: Is an item removed because it expired, became resolved, or became dominated? Give a different argument for each kind of removal. For histogram rectangles, specify exactly which boundary stops each height from extending.

11 Heaps and the candidate frontier

A heap gives efficient access to one extreme among changing candidates. It does not keep all elements in globally sorted iteration order. Decide which candidate you need to inspect next, then choose the heap direction.

To retain the largest k values of a stream, the only retained value a new arrival might replace is the smallest retained value. Use a min-heap of size at most k . After processing t inputs, it contains the largest $\min(k, t)$ values seen, counting duplicates.

```
pub fn kth_largest(nums: &[i32], k: usize) -> Option<i32> {
    use std::cmp::Reverse;
    use std::collections::BinaryHeap;
    if k == 0 || k > nums.len() {
        return None;
    }
    let mut best = BinaryHeap::with_capacity(k + 1);
    for &x in nums {
        best.push(Reverse(x));
        if best.len() > k {
            best.pop();
        }
    }
    best.peek().map(|x| x.0)
}
```

After insertion there are at most $k + 1$ candidates; deleting the smallest leaves exactly the best k among them. Previously discarded values cannot overtake retained values in an insertion-only stream. A deletion operation would invalidate that reasoning: a discarded value might become relevant again.

Time is $O(n \log(k + 1))$, with $O(k)$ auxiliary space. For a single static selection, sorting or quickselect may be preferable. A heap is particularly useful when answers are needed repeatedly as data arrives. Rust's `BinaryHeap` is a max-heap by default; `Reverse` reverses its ordering [S10](#).

When merging k sorted lists, the heap stores the first unconsumed item from each nonempty list. Every remaining item in a list is no smaller than that list's representative. Therefore the smallest representative is the smallest remaining item overall. Pop it and advance only its list. For N total items, time is $O(N \log(k + 1))$ and the frontier occupies $O(k)$ space.

Two heaps can maintain a median: all values in the lower half are at most all values in the upper half, and their sizes differ by at most one according to a chosen convention. Both the ordering condition and the size condition are required.

Practice: [LC 215 Kth Largest Element in an Array](#), [LC 703 Kth Largest Element in a Stream](#), [LC 347 Top K Frequent Elements](#), [LC 23 Merge k Sorted Lists](#), [LC 295 Find Median from Data Stream](#).

Check your understanding: Why this heap direction? Does an element represent a final selection or a frontier candidate? Could something discarded become useful after a deletion or priority change?

12 Sorting intervals and greedy choices

Sorting is a transformation: it makes relationships easier to reason about. Sorting intervals by start brings possible overlaps together. Sorting jobs by finish time can expose a safe scheduling choice. Sorting coordinates can turn a distance question into a question about neighbors.

To merge intervals, process starts in order and maintain the union of the processed intervals. The current last merged interval is the only existing interval a new interval can extend. Whether touching endpoints count as overlap depends on the endpoint convention. For closed intervals $[1, 2]$ and $[2, 3]$, merging is natural; for bookings occupying $[start, end)$, an end at time 2 releases the resource before a start at time 2.

For maximizing the number of nonoverlapping intervals, choose the available interval ending earliest. Suppose an optimal schedule begins with a different interval ending later. Replacing that first interval with the earliest-ending one leaves at least as much room for every later interval. The replacement preserves the schedule's size. Repeat on the remaining suffix. This exchange argument justifies the greedy choice.

```
pub fn max_nonoverlap(intervals: &mut [(i32, i32)]) -> usize {
    assert!(intervals.iter().all(|&(s, e)| s < e));
    intervals.sort_unstable_by_key(|&(s, e)| (e, s));
    let (mut last_end, mut count) = (None, 0);
    for &(start, end) in intervals.iter() {
        if last_end.is_none_or(|prev| start >= prev) {
            count += 1;
            last_end = Some(end);
        }
    }
    count
}
```

This uses half-open intervals and maximizes count. If intervals have different rewards, replacing one with an earlier-ending interval may lose value. Weighted scheduling needs a richer decision, commonly DP after sorting and finding compatible predecessors.

For meeting-room counts, sweep starts and ends while tracking simultaneous occupancy, or maintain a min-heap of active end times. Define tie ordering explicitly. A heap identifies which room frees first; it does not by itself prove a greedy scheduling objective.

Practice: [LC 56 Merge Intervals](#), [LC 57 Insert Interval](#), [LC 435 Non-overlapping Intervals](#), [LC 452 Minimum Number of Arrows to Burst Balloons](#), [LC 55 Jump Game](#), [LC 45 Jump Game II](#), [LC 1235 Maximum Profit in Job Scheduling](#).

Check your understanding: Can you replace a choice in an optimal solution without making the objective worse? Which property makes that replacement safe? Construct the smallest counterexample when the objective changes.

13 Linked lists and recursive trees

Linked-list problems often concern preserving access while rewiring edges. To reverse a list, maintain a reversed processed prefix and an untouched suffix. Save the next pointer before replacing the current pointer's next edge; otherwise the suffix may become unreachable. Moving pointers is bookkeeping around a reachability invariant.

Use a dummy head when the first node might change. For merging sorted lists, the output prefix contains exactly the smallest consumed elements, and each remaining list head is its smallest unconsumed element. For cycle questions, distinguish detecting a cycle from identifying its entrance.

Tree recursion becomes manageable when each call has a small contract. For height, an empty subtree has height zero and a node returns one plus the larger child height. For balance, a call can return either a valid height or a failure marker. That avoids recomputing heights at each ancestor.

For diameter, distinguish the value returned to the parent from the best answer found anywhere. The parent can extend only one downward branch, so return $\max(\text{left_height}, \text{right_height}) + 1$. A path whose highest node is the current node can use both children; its length in edges is $\text{left_height} + \text{right_height}$. Every simple path has a highest node, so considering this candidate at every node covers all paths.

For validating a binary search tree, a child must satisfy constraints inherited from every ancestor, not merely its parent. Pass lower and upper bounds, or prove that inorder traversal is strictly increasing under the no-duplicates convention. Use optional bounds rather than sentinel values that exclude valid extremes.

Traversal order reflects dependencies. Preorder processes before children; postorder combines child answers; inorder reveals sorted order in a BST; level order uses a queue for distance from the root. Traversing n nodes costs $O(n)$. Recursive depth is $O(\text{height})$, which is $O(n)$ for a chain and can overflow a language stack.

Practice: [LC 206 Reverse Linked List](#), [LC 21 Merge Two Sorted Lists](#), [LC 19 Remove Nth Node From End](#), [LC 104 Maximum Depth of Binary Tree](#), [LC 110 Balanced Binary Tree](#), [LC 98 Validate Binary Search Tree](#), [LC 543 Diameter of Binary Tree](#), [LC 236 Lowest Common Ancestor](#).

Check your understanding: What exactly does one call return? What information goes downward, and what comes upward? Is a returned branch extendable by the parent, or is it a completed path?

14 Backtracking and the search tree

Backtracking enumerates choices while maintaining a partial candidate. Define the state, available choices, stopping condition, and undo operation. A recursive call represents one node in the search tree, not necessarily one node of the input.

For subsets, at each input position decide whether to include it. There are 2^n subsets; copying all outputs takes $O(n \times 2^n)$ time in total. For permutations of n distinct values, choosing any unused value yields n factorial leaves and $O(n \times n \text{ factorial})$ output work.

The invariant is that the current path corresponds exactly to the choices made along the current recursion branch, and the used or remaining state agrees with it. After a recursive call returns, undo the mutation so siblings begin from the same parent state.

Pruning discards branches that cannot produce a valid or better result. With positive candidate values and a sum target, exceeding the target makes further additions useless. If negatives are allowed, that pruning is unsound. For optimization, a lower bound on future cost can justify pruning a branch whose best possible completion cannot beat the current answer.

Duplicate input values require distinguishing positions from output values. In sorted unique-permutation generation, skip an equal value when its previous equal value has not been used on the current path. This chooses one representative ordering for equivalent branches. Skipping all duplicate values would remove valid results that use repeated elements.

Memoization applies when different branches reach the same sufficient state and you only need the answer for that state. It is less direct when the requested output includes every distinct path, because those paths themselves matter. Caching must preserve the output contract.

Practice: [LC 78 Subsets](#), [LC 46 Permutations](#), [LC 47 Permutations II](#), [LC 39 Combination Sum](#), [LC 22 Generate Parentheses](#), [LC 79 Word Search](#), [LC 51 N-Queens](#).

Check your understanding: Which mutation must be undone? Is pruning logically impossible or merely unlikely? Are two paths reaching the same state interchangeable for the requested output?

15 Graph modeling and traversal

A graph is a representation of states and legal transitions. It may arrive as an edge list, or you may need to discover it inside a grid, lock combination, word transformation, or sequence of actions. Defining the state correctly is often harder than writing BFS.

DFS explores one branch deeply and is useful for reachability, components, and structural information. BFS processes states by the number of edges from the source. With equal-cost edges, the first discovery of a vertex gives its shortest distance. Mark a vertex when enqueueing it to prevent repeated insertion.

```
pub fn bfs_dist(adj: &[Vec<usize>], start: usize) -> Vec<Option<usize>> {
    use std::collections::VecDeque;
    assert!(start < adj.len());
    let mut dist = vec![None; adj.len()];
    let mut queue = VecDeque::from([start]);
    dist[start] = Some(0);
    while let Some(u) = queue.pop_front() {
        let next_distance = dist[u].unwrap() + 1;
        for &v in &adj[u] {
            if dist[v].is_none() {
                dist[v] = Some(next_distance);
                queue.push_back(v);
            }
        }
    }
    dist
}
```

The graph must use valid vertex indices. Time is $O(V + E)$, auxiliary space $O(V)$, excluding the adjacency representation. None means unreachable. For multi-source BFS, enqueue all sources with distance zero; this computes distance to the nearest source through a single shared expansion.

A visited set is only as correct as its state key. In a grid with collectible keys, position alone is insufficient: reaching the same cell with a different key set changes future moves. The state may be (row, column, key_mask). Distinct histories can merge only when their future possibilities and relevant costs agree.

Grid traversal must define adjacency: four directions or eight, boundaries, blocked cells, and whether revisiting is allowed. Flood fill asks reachability; shortest route asks distance. A DFS that reaches the destination first does not generally find the shortest route.

For an undirected cycle check, ignore the parent edge while detecting other visited neighbors; parallel edges need extra care. In a directed graph, use active versus completed states or topological processing. A single undifferentiated visited bit is not enough for every cycle test.

Practice: [LC 200 Number of Islands](#), [LC 133 Clone Graph](#), [LC 994 Rotting Oranges](#), [LC 127 Word Ladder](#), [LC 752 Open the Lock](#), [LC 864 Shortest Path to Get All Keys](#).

Check your understanding: What are the vertices? What makes an edge legal? Does revisiting the same location with different resources represent the same state? Are all transitions equally costly?

16 Dependencies and merging components

Topological order handles directed prerequisites. A vertex becomes ready when all incoming dependencies have been removed. Initialize a queue with zero-indegree vertices, process each, and decrement its outgoing neighbors. Every dequeued vertex has no unprocessed prerequisites.

If fewer than V vertices are processed, the remaining graph contains a directed cycle. Several valid orders may exist. A min-heap can choose the smallest available vertex if the output contract requires a lexicographically smallest order; a plain queue is enough for any valid order.

The same order can support DP. For longest paths in a DAG, propagate the best answer along outgoing edges once predecessors are settled. The graph need not look like a conventional network: increasing moves in a matrix create an acyclic dependency relation.

Union-find answers a different question: which elements currently belong to the same connected component as edges are added? Each set has a representative. Find follows parent links to that representative; union links two representatives. Path compression and union by size or rank make a sequence of operations nearly linear, with amortized $O(\alpha(n))$ per operation after initialization [S12](#).

The partition is the essential invariant: two elements share a representative exactly when the processed merges connect them. The internal parent tree is an implementation device, not necessarily a tree of original graph edges. A representative has no special semantic meaning unless you explicitly maintain one.

Ordinary union-find handles growing connectivity. It cannot directly undo arbitrary deletions. An offline problem may allow reversing time so deletions become insertions, or it may require rollback machinery. Directed reachability is also not captured by merely merging edge endpoints.

Practice: [LC 207 Course Schedule](#), [LC 210 Course Schedule II](#), [LC 329 Longest Increasing Path in a Matrix](#), [LC 684 Redundant Connection](#), [LC 721 Accounts Merge](#), [LC 547 Number of Provinces](#).

Check your understanding: Is the relation directed precedence or undirected connectivity? Can groups split later? Which metadata must be combined when components merge?

17 Weighted shortest paths

Choose a shortest-path method from the edge costs and state structure. Equal-cost edges permit BFS. Costs restricted to zero and one permit 0-1 BFS using a deque. Nonnegative costs permit Dijkstra. A DAG permits processing in topological order even when edges are negative. General negative edges require another method, such as Bellman-Ford, with attention to relevant negative cycles.

Dijkstra keeps tentative distances and expands the smallest currently known unsettled distance. A path through an unsettled vertex cannot improve the smallest tentative distance through negative future cost because all edge costs are nonnegative. That is the central condition behind finalizing a distance [S11](#).

With a binary heap and duplicate entries, relaxing an edge inserts a new distance rather than updating the old heap entry. When popping, skip entries that no longer equal the best recorded distance. Marking a node permanently visited when it is first inserted is wrong: a later relaxation may improve it.

For a typical adjacency-list implementation, a safe bound is $O((V + E) \log(V + E))$ time and $O(V + E)$ auxiliary storage with duplicate heap entries. Stronger familiar bounds use assumptions about graph representation and edge multiplicity. State the version you actually implement.

For a simple counterexample to first-discovery finalization, take edges S to A of cost 10, S to B of cost 1, and B to A of cost 1. The first route discovered to A costs 10; the shortest route costs 2.

Constraints can enlarge the state. If you may use only a limited number of edges, a cheap arrival with no remaining budget does not necessarily dominate a more expensive arrival with budget left. Track enough information, such as (vertex, edges_used). For "at most k stops," translate carefully to at most k + 1 edges.

Practice: [LC 743 Network Delay Time](#), [LC 1631 Path With Minimum Effort](#), [LC 1368 Minimum Cost to Make at Least One Valid Path in a Grid](#), [LC 787 Cheapest Flights Within K Stops](#). In minimum-effort paths, extending a path combines cost with max rather than addition; justify that variant's monotone path-cost behavior.

Check your understanding: What does a distance measure? Why is a popped candidate final? Does the visited key include every resource that changes future feasibility? Is a node unreachable or merely not yet improved?

18 Dynamic programming from sufficient state

Dynamic programming reuses answers for repeated subproblems. Start by describing the choices in a correct recursive solution. Then ask which parts of its history determine the remaining choices and their value. Those parts form the state.

For House Robber, let $\text{best}(i)$ mean the largest sum obtainable from the first i houses without choosing adjacent houses. A valid solution either skips the last house, giving $\text{best}(i - 1)$, or takes it, giving $\text{best}(i - 2) + \text{value}[i - 1]$. These cases cover all possibilities and do not overlap in their final decision.

```
pub fn rob(values: &[i32]) -> i64 {
    assert!(values.iter().all(|&x| x >= 0));
    let (mut two_back, mut one_back) = (0_i64, 0_i64);
    for &value in values {
        let current = one_back.max(two_back + i64::from(value));
        two_back = one_back;
        one_back = current;
    }
    one_back
}
```

Before processing the next house, `one_back` is the optimum for all processed houses and `two_back` is the optimum without the most recent one. The update implements the recurrence exactly. Time is $O(n)$, space $O(1)$. Derive the full state before compressing storage; saving memory too early makes dependencies harder to see.

Every DP explanation should specify five things: state meaning, transitions, base cases, evaluation order, and answer location. "Use a 2D array" specifies none of them.

Unreachable states need a representation that cannot be mistaken for a valid score. In a minimum-cost DP, initializing every state to zero invents free solutions. Prefer Option or a carefully guarded infinity. Do not add costs to an infinity sentinel without considering overflow.

Top-down memoization evaluates reachable states on demand and may closely match the recurrence. Bottom-up DP chooses an explicit dependency order and avoids recursive stack depth. Both need the same sufficient state. A cyclic recurrence is not automatically solved by memoization; cycles may require a different formulation or algorithm.

The cost is roughly number of evaluated states times work per state, plus input and output work. A state indexed by position and budget might have $O(nB)$ states; if each tries B choices, the cost becomes $O(nB^2)$.

Practice: [LC 70 Climbing Stairs](#), [LC 198 House Robber](#), [LC 746 Min Cost Climbing Stairs](#), [LC 322 Coin Change](#), [LC 139 Word Break](#).

Check your understanding: Can two histories with the same state have different legal futures? If so, the state is incomplete. Does each transition move toward an already solved or strictly smaller subproblem?

19 Dynamic programming families

For sequence alignment, a useful state is a pair of prefix lengths. Longest Common Subsequence asks what can be achieved using the first i elements of one sequence and the first j of another. Equal final elements can extend a shorter common subsequence; otherwise one final element must be skipped. Edit Distance uses a similar grid but different operations and base cases. Similar table shapes do not imply identical recurrences.

For 0/1 subset sum, state `reachable[s]` records whether a processed subset reaches sum s . Updating sums downward ensures the current item is used at most once. Updating upward allows the newly reached state to reuse that same item, which corresponds to a different, unbounded problem.

For counting coin combinations, processing each coin outside the sum loop imposes a consistent order on selections. Processing target sums outside and trying every final coin often counts ordered sequences instead. Derive the counted objects before choosing loop order.

For interval DP, define what happens inside $[l, r]$ and select a split or a last action. Choosing the last action can make its neighbors known and separate subproblems; choosing the first action may leave complicated interactions. State a measure, such as interval length, that decreases in recursive calls.

For tree DP, a child may need to return several answers conditional on the parent's decision. In House Robber III, return best values when the current node is taken and when it is skipped. Taking it forces both children to be skipped; skipping it permits the better choice independently in each child subtree.

For bitmask DP, the set of visited or selected elements is encoded by a small integer. A traveling-state formulation may use $(\text{mask}, \text{last_vertex})$. There are $O(n \text{ times } 2 \text{ to the } n)$ states and potentially $O(n \text{ squared times } 2 \text{ to the } n)$ transition work. Small n is essential.

Longest Increasing Subsequence has both a quadratic DP and an $O(n \log n)$ method. The faster method stores the smallest possible tail for each subsequence length. Those tails need not belong to one actual subsequence together; they summarize the best extension opportunities. Smaller tails dominate larger tails of the same length.

Practice: [LC 1143 Longest Common Subsequence](#), [LC 72 Edit Distance](#), [LC 416 Partition Equal Subset Sum](#), [LC 518 Coin Change II](#), [LC 494 Target Sum](#), [LC 312 Burst Balloons](#), [LC 337 House Robber III](#), [LC 300 Longest Increasing Subsequence](#).

Check your understanding: Are you optimizing, counting, or deciding existence? Are objects ordered? May an item be reused? Which dependency would an in-place update overwrite too early?

20 Tries and string structure

A trie stores shared prefixes explicitly. Each edge consumes one symbol; terminal markers distinguish complete words from prefixes. After consuming a query prefix, the current node represents exactly the stored words that begin with it.

Lookup time is $O(m)$ for a word of m symbols when child lookup is constant time. Space depends on all stored prefixes and the child representation. Fixed alphabet arrays trade space for predictable access; maps support larger alphabets with allocation and lookup overhead.

Wildcard search branches over children at wildcard positions, so it can explore many nodes. It is not generally $O(m)$. Combining a trie with grid backtracking lets many candidate words share prefix checks. The grid's visited state must still be restored between branches.

String matching algorithms reuse partial agreement. In KMP, the prefix function records the longest proper prefix that is also a suffix of a processed prefix. After a mismatch, this tells you how much matched structure can survive without rechecking all earlier text. The useful idea is the fallback relation between borders; memorize code only after tracing the relation.

Rolling hashes summarize substrings for fast comparisons, but equal hashes do not prove equal strings unless collisions are excluded by the method or matches are verified. A correctness explanation must account for collisions rather than treating a hash as a unique identity.

Palindrome problems often become simpler by changing the center of attention. Expanding around each character and gap handles odd and even lengths in $O(n^2)$ time with $O(1)$ auxiliary space. A DP or specialized linear algorithm may be appropriate for different constraints. The requested output determines whether finding, counting, or partitioning palindromes is the real task.

Practice: [LC 208 Implement Trie](#), [LC 211 Design Add and Search Words](#), [LC 212 Word Search II](#), [LC 28 Find the Index of the First Occurrence in a String](#), [LC 5 Longest Palindromic Substring](#), [LC 131 Palindrome Partitioning](#).

Check your understanding: What is one symbol: byte, Unicode scalar, or user-perceived character? Is a prefix a complete word? What information survives a mismatch? Does a hash comparison need verification?

21 Bits mathematics and small state spaces

Bit operations are useful when a compact set or algebraic property matches the problem. XOR cancels equal values because $x \text{ XOR } x = 0$, and combining with zero preserves x . In Single Number, every duplicated value cancels, leaving the value with odd multiplicity. The proof depends on the promised multiplicities.

A bitmask can encode a subset of a small universe. Bit i records membership of element i . Setting, clearing, and testing a bit implement set operations. Enumerating all masks still costs exponential time; compact representation does not remove the number of states.

The expression $x \& (x - 1)$ clears the lowest set bit for a positive unsigned integer. Treat zero and subtraction carefully in Rust. Shifts must stay within the integer type's width. Signed right shifts and overflowing arithmetic can change the intended mathematics.

Mathematical transformations often remove simulation. Euclid's algorithm uses $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$ for positive b . Exponentiation by squaring exploits the decomposition of an exponent into powers of two. Modular arithmetic groups values by residue, but negative remainders require a normalized convention.

Counting problems need an explicit model. Decide whether order matters, whether repetitions are allowed, and whether objects are distinguishable. A binomial coefficient counts choices of positions without order; permutations count arrangements. The wrong combinatorial model can produce a tidy formula for the wrong objects.

Numeric constraints matter independently of algorithmic complexity. A linear-time algorithm can overflow while summing n values or multiplying two coordinates. Widen before the operation, not after. If an answer is required modulo M , apply modular arithmetic where algebraically valid; division needs an inverse and the inverse may not exist.

Practice: [LC 136 Single Number](#), [LC 191 Number of 1 Bits](#), [LC 338 Counting Bits](#), [LC 50 Pow](#), [LC 78 Subsets](#), [LC 201 Bitwise AND of Numbers Range](#).

Check your understanding: Which identity makes the trick correct? What input promise does it rely on? Would repeated values three times instead of twice change the argument?

22 Range queries under updates

A prefix array is ideal for static sums but one point update changes many later prefixes. A Fenwick tree stores overlapping blocks of accumulated values so both point additions and prefix-sum queries take $O(\log n)$. Internally, one-based indexing makes the lowest set bit identify a block's size.

To query a prefix, repeatedly add the stored block ending at the current index and remove its lowest set bit. Those blocks partition the queried prefix. To add at one position, walk upward through the blocks that contain it. Index zero requires special handling; an update loop that starts at zero and adds its lowest set bit never advances.

A segment tree partitions the index range recursively. Each node summarizes a segment, and a query combines the summaries of a small collection of disjoint segments. The combine operation must be associative, and empty contributions need an identity. Examples include sum with identity zero, minimum with an appropriate infinity, and maximum with negative infinity.

Point updates and range queries take $O(\log n)$ for a standard balanced segment tree, with $O(n)$ storage. Lazy propagation extends this to supported range updates by recording deferred actions. It also requires a correct rule for composing actions and applying them to summaries; "store a lazy value" is not a complete design.

Coordinate compression replaces ordered values by ranks when only their order matters. It preserves comparisons, not numeric distances. If an algorithm uses actual gaps between coordinates, retain those gaps separately.

Before building an advanced structure, check whether the queries are offline, the data is static, or only a small subset of operations is required. Sorting queries, prefix sums, or a monotonic deque may solve a simpler contract with less machinery.

Practice: [LC 307 Range Sum Query Mutable](#), [LC 315 Count of Smaller Numbers After Self](#), [LC 327 Count of Range Sum](#), [LC 218 The Skyline Problem](#). The latter problems admit multiple approaches; compare what information each approach maintains.

Check your understanding: What is stored at one node? Why can query pieces be combined in that order? Is the update assignment or addition? Does compression preserve the operation you intend to perform?

23 Data structure design and composed invariants

Design problems ask for a set of operations with target costs. Write the operation table first. An LRU cache needs lookup by key, promotion to most recently used, insertion, and eviction of the least recently used entry. A map locates entries; a doubly linked order supports constant-time removal and insertion once an entry is located.

The important invariant spans both structures: every live key has exactly one order node, every order node has exactly one map entry, and the order agrees with recency. Updating one structure without the other creates a bug even if each structure is locally well formed.

A randomized set combines an array for uniform index selection with a map from value to array index. To delete, swap the final value into the removed position, update that moved value's index, and pop. The invariant is that the map and array are exact inverses for all live values. Forgetting the moved value's map update corrupts later deletions.

A minimum stack can store both the current value and the minimum through that stack depth. Duplicated minimum values must remain correct after one is popped. A queue built from two stacks has amortized constant-time operations because each element moves between stacks at most once before removal.

In Rust, an arena of nodes addressed by indices can make ownership simpler than a graph of shared mutable pointers. Reusing slots may need generation counters if stale handles can escape. Pick a representation that supports the required contract; implementation convenience cannot replace the operation-cost argument.

Practice: [LC 146 LRU Cache](#), [LC 380 Insert Delete GetRandom O\(1\)](#), [LC 155 Min Stack](#), [LC 232 Implement Queue using Stacks](#), [LC 981 Time Based Key Value Store](#).

Check your understanding: Which invariant connects the structures? What happens on replacement of an existing key? What are the empty and capacity-zero behaviors? Is the promised cost worst-case, expected, or amortized?

24 Rust as an implementation tool

Keep algorithm design and Rust mechanics distinct enough to diagnose errors. First state the algorithm in plain language. Then choose ownership, numeric types, and collections that preserve that meaning.

Need	Usual Rust representation	Detail to remember
Dense indexed state	<code>Vec<T></code>	Indices use <code>usize</code> ; subtraction can underflow.
Membership or counts	<code>HashSet</code> or <code>HashMap</code>	Hash lookup costs include constructing and hashing the key.
Ordered keys	<code>BTreeSet</code> or <code>BTreeMap</code>	Use when order queries are required.
FIFO traversal	<code>VecDeque</code>	<code>pop_front</code> avoids shifting the remaining elements.
LIFO work	<code>Vec</code>	<code>push</code> and <code>pop</code> express the stack directly.
Changing maximum	<code>BinaryHeap</code>	Iteration is not sorted order.
Changing minimum	<code>BinaryHeap<Reverse<T>></code>	Tuple ordering also controls tie breaks.
Missing or unreachable	<code>Option<T></code>	Keep absence distinct from a valid zero.

The standard collection documentation describes their guarantees and tradeoffs [S13](#). For heap direction and ordering, see [S10](#).

Use `i64` for accumulated `i32` sums when constraints warrant it. Cast operands before addition or multiplication. A product computed as `i32` and then cast to `i64` has already overflowed. `usize` is suitable for valid array positions; it is awkward for `-1` sentinels. A half-open interval often removes that sentinel entirely.

Strings are UTF-8. For guaranteed ASCII input, `s.as_bytes()` gives convenient indexed symbols. For Unicode scalar values, collect `s.chars()` if indexed access is needed; this allocates and still does not represent grapheme clusters. State the alphabet assumption instead of silently treating bytes as characters [S14](#).

Borrowing issues often reveal overlapping access patterns. Copy a small scalar from a map or heap before mutating the collection. For counts, `entry(key).or_insert(0)` gives a direct update path. Avoid cloning whole graphs or paths merely to silence a borrow error without including the cost in the analysis.

Rust recursion can exhaust the process stack on a long chain. An explicit stack provides more predictable memory use for deep graph traversal. Shared tree judges may use `Rc<RefCell<TreeNode>>`; index-based adjacency lists can be simpler for algorithms you control. Neither representation changes the proof.

When testing, distinguish compiler correctness, sample correctness, and algorithmic correctness. Build a slow reference for small inputs when practical. Enumerate or generate small cases, compare answers, and inspect the smallest disagreement. Tests search for counterexamples; the invariant explains why all permitted inputs work.

25 Similar looking problems with different rules

The fastest way to overfit a pattern is to ignore the condition that made it work. Use these pairs as deliberate comparisons. Explain the changed assumption before studying the second solution.

First problem or condition	Changed condition	What must be reconsidered
Positive-sum shortest window, LC 209	Negative values allowed, LC 862	Sum monotonicity; prefix sums with candidate dominance
Static range sums, LC 303	Point values change, LC 307	Precomputed prefixes versus maintained aggregates
Any matching pair, LC 1	Unique triplets, LC 15	Enumeration, duplicate rules, and sorted-pair elimination
Unweighted shortest path	Nonnegative unequal weights, LC 743	BFS discovery order versus distance-priority expansion
Cheapest unrestricted route	Limited stops, LC 787	Vertex alone versus vertex and remaining budget
Maximum number of intervals, LC 435	Weighted reward, LC 1235	Earliest finish exchange proof no longer preserves value
Each item at most once, LC 416	Reusable coins, LC 518	State meaning and update direction
Coin combinations, LC 518	Ordered sums, LC 377	Whether different orders count as different answers
Subsequence, LC 1143	Contiguous substring, LC 718	Skipping is allowed in one recurrence, forbidden in the other
Reachability in a grid, LC 200	Keys change moves, LC 864	Position alone is no longer a sufficient state
Append-only kth largest, LC 703	Arbitrary deletions	Discarded values may become relevant again
Equal-looking strings	Hashes happen to match	A hash equality needs collision reasoning

These are conceptual comparisons, not claims that every pair differs in only one detail. Read both contracts, including output conventions and constraints.

An invariant must also be specific enough to prove the claimed result. "The stack is sorted" does not explain why popped candidates are unnecessary. "The DP stores the best answer" does not define the subproblem. "Every finalized distance is shortest" needs the graph assumptions and a finalization rule.

Use three challenges on each learned technique: change the input restrictions, change the requested objective, and change the supported operations. Positives to negatives, existence to counting, and insertions to deletions expose different kinds of hidden assumptions.

26 A curriculum built around dependencies

This sequence is a proposed study route, not an empirically optimal schedule. Advance when you can explain and implement the core ideas, and revisit earlier topics in mixed practice. A difficult problem is a later checkpoint, not a prerequisite for continuing every other chapter.

Stage	Learn and practice	Readiness evidence
A	Complexity, arrays, hashing, sorting; LC 1, 242, 49, 121	Describe a correct baseline and remove a repeated lookup or scan.
B	Prefix sums, pointers, windows; LC 303, 560, 167, 209, 3	Explain safe endpoint movement and identify when it fails.
C	Binary search, stacks, heaps; LC 35, 875, 20, 739, 215	State the search boundary or candidate invariant precisely.
D	Lists, recursive trees, backtracking; LC 206, 104, 98, 78, 46	Define a recursive contract and restore branch-local state.
E	Graph modeling, BFS, DFS, dependencies; LC 200, 994, 207	Define sufficient state and justify traversal order.
F	DP and greedy proofs; LC 198, 322, 416, 1143, 435	Derive a recurrence or a valid exchange argument.
G	Compositions and changed assumptions; LC 239, 862, 743, 787	Explain why a familiar simpler technique is insufficient.
H	Optional depth; tries, range structures, bitmasks, advanced strings	Match the extra machinery to a concrete operation or constraint.

For each family, choose one clear anchor, one variation, and one problem attempted without its topic label. The anchor teaches the idea. The variation changes a condition. The untagged attempt tests whether you can select the idea yourself.

Suggested first pass: LC 1, 242, 49, 121, 303, 560, 167, 209, 3, 35, 875, 20, 739, 215, 206, 104, 78, 200, 198, and 435. This is a starting selection, not a minimum or maximum quota. If a prerequisite is missing, study it explicitly. If an exercise is routine, move to a meaningful variation.

Once you know several families, mix them. Keep selected evaluation problems unseen: do not browse their reference implementations while annotating the corpus. Familiar problems measure retention; unfamiliar problems help measure transfer. Track hint level, correctness argument, implementation reliability, and whether the approach survives changed assumptions.

Commercial and community curricula are useful indexes. NeetCode emphasizes grouped practice and review [S1](#); Grind 75 explains its selection and mixed ordering [S2](#); Sean Prashad offers approach-selection heuristics [S3](#). Use one primary sequence and borrow extra problems to address a specific gap.

27 Transfer exercises

These original exercises test ideas from the book in different settings. Their technique names are deliberately omitted here. Work from the contracts. Chapter 28 contains staged hints and reasoning, so keep it closed during an independent attempt.

Exercise A Parcel ledger. A ledger contains n signed integer balance changes. Count the nonempty consecutive stretches whose total is exactly t . For $[2, -2, 2]$ and $t = 2$, return 3. Assume all accumulated sums and the count fit i64. Seek expected $O(n)$ time.

Exercise B Radio buffer. Positive packet sizes arrive in order. Find the fewest consecutive packets whose total is at least a positive capacity c , or report no such stretch. For $[4, 1, 2, 6]$ and $c = 7$, return 2. Then decide whether your algorithm still works if packet sizes can be negative adjustments.

Exercise C Exhibit scores. Scores arrive one at a time, duplicates included. After at least k arrivals, report the k th-largest score so far. For $k = 3$ and arrivals $[8, 2, 8, 5]$, the last two reports are 2 and 5. Use $O(k)$ retained values. Explain what breaks if arbitrary old scores can be deleted.

Exercise D Printer capacity. A machine processes positive job sizes in their given order. A day handles a consecutive group with total size at most capacity c , and jobs cannot be split. Find the smallest capacity that finishes within d days, where $1 \leq d \leq n$. For $[3, 2, 4, 1]$ and $d = 2$, return 5. Explain your decision test and search bounds.

Exercise E Gallery visits. Each exhibit occupies a half-open time interval $[start, end)$, with $start < end$. Choose the largest number of nonoverlapping exhibits. For $[0, 4), [1, 2), [2, 3), [3, 5)$, return 3. Then give each exhibit a reward and explain whether the same rule maximizes reward.

Exercise F Maintenance corridor. Rooms form an unweighted undirected graph. Several rooms contain maintenance stations. For every room, find the fewest edges to any station, or report unreachable. In the path 0-1-2-3-4 with stations 0 and 4, return $[0, 1, 2, 1, 0]$.

Exercise G Cooling schedule. Each day offers a nonnegative reward, but accepting on one day prevents accepting on the next. Find the largest total reward. For $[4, 7, 2, 9]$, return 16. Then change the rule so accepting a reward blocks the next two days and derive a new recurrence.

Exercise H Museum door. A graph has doors that require particular keys, and keys are acquired at vertices without being consumed. Find the minimum number of moves from a start to a target. Explain why marking only the room as visited can discard a necessary route. You may assume at most six distinct keys.

For every exercise, submit five things to yourself: a baseline, the key observation, an invariant or recurrence, complexity, and a counterexample to a tempting incorrect approach. A correct output on the example is only the beginning.

28 Hints and transfer exercise answers

A, small hint: compare two accumulated totals. The difference between prefixes is a range sum. Maintain the number of earlier prefix values equal to $\text{current_prefix} - t$, querying before inserting the current prefix. Seed the empty prefix once. Expected $O(n)$ time and $O(n)$ space. The sample's valid stretches are each singleton 2 and the full ledger. A positive-sum window is not justified with signed changes.

B, small hint: examine how removing the oldest packet changes the total. With positive sizes, expand right and shrink while sufficient, recording every valid candidate before removing its left value. $O(n)$ time and $O(1)$ auxiliary space. The stretch $[1, 2, 6]$ can shrink to $[2, 6]$. Negative adjustments break this monotone sum rule; $[1, -1, 5]$ at capacity 5 demonstrates the failure.

C, small hint: which retained score is easiest for a new score to replace? Keep the largest k arrivals in a min-heap. The root is the k th largest once the heap has k elements. Each insertion costs $O(\log(k + 1))$ with $O(k)$ storage. Arbitrary deletion may promote a formerly discarded score, so keeping only k values is insufficient for that expanded contract.

D, small hint: test one proposed capacity before optimizing it. Greedily place as many consecutive jobs as fit each day; this uses the fewest days for that capacity because no valid first day can end later, and the argument repeats on the suffix. Feasibility is monotone as capacity increases. Search from $\max(\text{job})$ through $\sum(\text{jobs})$. A check costs $O(n)$; binary search costs $O(n \log R)$, where R is the number of integer capacities in the range. Capacity 5 permits $[3, 2]$ and $[4, 1]$.

E, small hint: preserve as much future time as possible. Choose the earliest-finishing compatible exhibit. An exchange of the first choice in an optimal schedule proves safety, followed by induction. Sorting dominates at $O(n \log n)$. Weighted rewards break the count-preserving exchange: an interval $[0, 4)$ worth 100 beats three shorter compatible intervals worth 1 each. Use a recurrence comparing skip versus take plus the best compatible prefix.

F, small hint: let all stations start searching together. Initialize a BFS queue with every distinct station at distance zero. First discovery records distance to the nearest station because layers increase by one edge. Time $O(V + E)$, space $O(V)$, excluding the graph. Separate BFS runs are unnecessary repeated work. Rooms in a component with no station remain unreachable.

G, small hint: classify solutions by the final day's decision. For the first i days, $\text{best}(i) = \max(\text{best}(i - 1), \text{reward}[i - 1] + \text{best}(i - 2))$, with $\text{best}(j) = 0$ for $j \leq 0$. Blocking two following days changes $i - 2$ to $i - 3$. Both cost $O(n)$ time with constant storage after deriving the dependency. Selecting each locally larger adjacent reward can miss a better combination.

H, small hint: two visits to the same room may permit different next moves. Use state (room, key_mask) and BFS because each move costs one. Entering a room updates the key mask; traversing a locked edge requires its key. There are at most V times 2 to the K states. Scanning adjacency for each mask gives $O((V + E) \text{ times } 2 \text{ to the } K)$ time and $O(V \text{ times } 2 \text{ to the } K)$ distance storage. A room reached before collecting a key must be revisitable with that key.

These exercises deliberately reuse established ideas. Their value comes from reconstructing and justifying the approach without a technique label. Solving one newly worded example alone does not demonstrate broad transfer.

29 Review pages and contribution method

Use this blank record after a meaningful attempt. Keep the answer hidden on subsequent reviews until you have tried to retrieve it.

Field	Your notes
Problem and date	
Independent attempt or assistance received	
Straightforward correct approach	
Repeated or unnecessary work	
Observation that removes it	
Assumptions required	
State or invariant	
Why updates preserve correctness	
Time and space costs	
Smallest counterexample to my first idea	
Implementation mistake	
Changed-condition problem to try	
Delayed review result	

When contributing a new explanation, include the contract, a derivation, the invariant or recurrence, complexity with assumptions, a failure boundary, and a practice link. Link to source statements and respect their licenses. Explain alternative approaches when they teach a distinct decision, rather than presenting several implementations without a comparison.

Keep the corpus index separate from mastery records. A file existing in the repository does not mean the reader understands it, nor does a passing example establish a complete correctness proof. A full-corpus edition should preserve multiple valid approaches and distinguish reviewed explanations from automatically suggested annotations.

When adding a problem family, test the explanation against both a positive example and a counterexample. When adding an original exercise, check examples, edge cases, and the reference algorithm against a simple independent implementation on small inputs where feasible. A changed story must still have a precise contract.

30 Advanced families and when they become useful

The main chapters cover a broad foundation. This directory identifies additional families and the prerequisite observation that makes each worth studying. It is a map for further learning, not a claim that every technique can be mastered from a table.

Family	When to investigate it	Core idea or proof obligation
Divide and conquer	Cross-part interactions can be processed efficiently	Solve smaller parts and account for every interaction crossing the split.
Merge-sort counting	Count ordered pairs with a value inequality	Sorted halves permit a monotone cross-counting scan before merging.
Sweep line	Eligibility changes at ordered coordinates or times	Apply events consistently, including ties, while maintaining the active set.
Meet in the middle	Subset enumeration is too large but half-enumeration fits	Combine compatible partial solutions without losing constraints.
Digit DP	Count numbers in a large bound with digit restrictions	Track position, tightness to the bound, and sufficient digit history.
State-machine DP	Actions depend on a small mode such as holding a stock	Define legal transitions and whether state updates use yesterday or today.
Rerooting DP	Need a tree answer for every possible root	Combine information from children and from the parent side without double counting.
Binary lifting	Repeated ancestor or jump queries	Precompute jumps of powers of two and compose the bits of a distance.
Euler tours	Subtree membership must become a range query	Entry and exit times give a contiguous subtree interval under a fixed root.
Minimum spanning trees	Connect all vertices with minimum total edge weight	Use a cut or exchange argument; shortest routes are a different objective.
Strongly connected components	Directed cycles should be treated as units	Contract mutual-reachability components to obtain a DAG.
Bridges and articulation points	Identify edges or vertices critical to connectivity	Low-link information tracks reachability outside the DFS parent route.
Eulerian traversal	Use every edge exactly once	Establish degree and connectivity conditions; distinguish edges from vertices.
All-pairs shortest paths	Many source-target queries on a modest graph	Floyd-Warshall progressively permits intermediate vertices; $O(V^3)$.
Maximum flow and matching	Capacity-constrained assignments or disjoint routes	Residual edges allow earlier choices to be revised; prove the reduction.
0-1 BFS	Edge weights are exactly zero or one	Push zero-cost improvements to the front and one-cost ones to the back.

Family	When to investigate it	Core idea or proof obligation
KMP and Z functions	Repeated prefix agreement appears in strings	Reuse verified matches rather than restarting at every position.
Manacher's algorithm	Need palindrome radii in linear time	Reuse mirrored information within a known palindrome and extend safely.
Sparse tables	Static idempotent range queries, such as minimum	Precompute powers of two; overlapping blocks are safe for idempotent operations.
Monotone optimizations for DP	A recurrence has extra order or convexity structure	Prove the required property; a familiar DP shape alone is insufficient.
Game DP	Players alternate optimal choices	State the current player's value and how a move changes perspective.
Geometry	Orientation, containment, or spatial ordering is central	Define exact predicates, degeneracies, and numeric precision requirements.

For digit DP, leading zeros may mean "no digit chosen yet" rather than actual zero digits. Tightness records whether the chosen prefix equals the bound so far; once below the bound, later digits are less constrained. Memoization keys must include every property affecting future digit choices.

For game DP, a score-difference state often simplifies alternating turns: current advantage equals chosen gain minus the opponent's best advantage on the remaining state. This relies on the game's rules and zero-sum objective. Do not apply it blindly to cooperative or non-zero-sum games.

For minimum spanning trees, a globally cheap connection structure need not give shortest paths from any source. The graph could have a shortest direct route that a spanning tree omits. Ask whether the objective sums selected edges once or sums travel cost along routes.

For divide and conquer, include merge work in the recurrence. Merge-sort-based pair counting commonly costs $O(n \log n)$, while a quadratic combine step defeats the desired improvement. When counting inequalities such as $a[i] > 2 * a[j]$, widen before multiplying and preserve the original index ordering through the split.

Advanced practice, chosen by need: [LC 493 Reverse Pairs](#), [LC 1851 Minimum Interval to Include Each Query](#), [LC 1755 Closest Subsequence Sum](#), [LC 2035 Partition Array Into Two Arrays to Minimize Sum Difference](#), [LC 2376 Count Special Integers](#), [LC 309 Best Time to Buy and Sell Stock with Cooldown](#), [LC 834 Sum of Distances in Tree](#), [LC 1483 Kth Ancestor of a Tree Node](#), [LC 1584 Min Cost to Connect All Points](#), [LC 1192 Critical Connections in a Network](#), [LC 332 Reconstruct Itinerary](#), [LC 486 Predict the Winner](#), [LC 1392 Longest Happy Prefix](#).

Use external references for a full derivation before implementing an unfamiliar advanced technique. The CP-Algorithms index [S16](#) is a broad reference. Study the theorem or invariant that makes an optimization valid, and keep a simpler correct implementation for small-case comparison when feasible.

31 Sources and further reading

The algorithm explanations in this book are original teaching treatments of standard techniques. The sources below provide curricula, formal algorithm references, language guarantees, and research context. They do not validate a specific count of patterns or guarantee interview outcomes.

S1. NeetCode, How to Prepare for Coding Interviews. A practitioner guide emphasizing fundamentals, grouped pattern practice, and repetition. Its preparation advice is not a controlled study. <https://blog.neetcode.io/p/prepare-coding-interviews>

S2. Yangshun Tay, Grind 75 FAQ. Explains question selection, topic coverage, and mixed practice. Useful as a curriculum rationale, with no guarantee that one list is sufficient for every learner. <https://www.techinterviewhandbook.org/grind75/faq>

S3. Sean Prashad, Leetcode Patterns. A free practice index and collection of approach-selection heuristics. Treat a heuristic as a hypothesis whose assumptions still need checking. <https://seanprashad.com/leetcode-patterns/>

S4. USACO Guide, How to Practice. Collects differing advice from competitive programmers about difficulty, editorials, implementation, and productive effort. <https://usaco.guide/general/practicing>

S5. Margulieux, Morrison, and Decker, 2020. Reducing withdrawal and failure rates in introductory programming with subgoal labeled worked examples. A semester-long quasi-experiment with 265 students found improved initial quiz performance and fewer failures or withdrawals, without a significant improvement in average exam scores. This supports meaningful step explanations with limits on claims about durable mastery. <https://doi.org/10.1186/s40594-020-00222-7>

S6. Roediger and Karpicke, 2006. Test-Enhanced Learning. Retrieval practice improved delayed retention relative to repeated study in experiments using prose passages. Applying the principle to algorithm reconstruction is an extrapolation. https://learninglab.psych.purdue.edu/downloads/2006/2006_Roediger_Karpicke_PsychSci.pdf

S7. Rohrer, Dedrick, Hartwig, and Cheung, 2020, published online 2019. A Randomized Controlled Trial of Interleaved Mathematics Practice. A study of 54 seventh-grade classes found higher delayed test scores with interleaved practice. The setting was school mathematics, not LeetCode. <https://files.eric.ed.gov/fulltext/ED595322.pdf>

S8. Gick and Holyoak, 1983. Schema Induction and Analogical Transfer. Comparing analogous examples helped participants extract an underlying schema associated with transfer. This motivates explicit comparisons between problems; it does not establish an automated corpus taxonomy. [https://doi.org/10.1016/0010-0285\(83\)90002-6](https://doi.org/10.1016/0010-0285(83)90002-6)

S9. CP-Algorithms, Binary Search. Formalizes monotone predicates, boundary invariants, and searching over answers. https://cp-algorithms.com/num_methods/binary_search.html

S10. Rust standard library, BinaryHeap. Documents ordering, Reverse, operations, and complexity guarantees. <https://doc.rust-lang.org/std/collections/struct.BinaryHeap.html>

S11. CP-Algorithms, Dijkstra. Gives the nonnegative-edge condition and the correctness argument for distance finalization. <https://cp-algorithms.com/graph/dijkstra.html>

S12. CP-Algorithms, Disjoint Set Union. Explains representatives, path compression, union by size or rank, and amortized complexity. https://cp-algorithms.com/data_structures/disjoint_set_union.html

S13. Rust standard library, Collections. Describes collection choices and operation costs.

<https://doc.rust-lang.org/std/collections/index.html>

S14. The Rust Programming Language, Storing UTF-8 Encoded Text with Strings. Explains bytes, scalar values, indexing, and string representation.

<https://doc.rust-lang.org/book/ch08-02-strings.html>

S15. AlgoMonster, algorithm selection flowchart and curriculum. A source of selection prompts and technique families. The flowchart is a guide to candidate approaches, not a universal correctness procedure. <https://algo.monster/flowchart>

S16. CP-Algorithms, algorithm reference index. A directory of advanced graph, string, numeric, and data structure techniques. Follow each technique's detailed assumptions and proof before using it. <https://cp-algorithms.com/>

The practice methods in Chapters 1 and 26 are this book's recommendations informed by these sources. Comparing related examples is motivated by S8; explaining functional steps is motivated by S5. A reader's delayed independent solutions are the evidence that the recommendations are working for that reader.

32 Selected problem index

The following index links every LeetCode problem referenced in the book to its original statement. Numbers identify problems; they are not a required solving order. The source repository also includes a local implementation map. Open solution code after attempting the problem.

ID	Original problem statement
1	Two Sum
3	Longest Substring Without Repeating Characters
5	Longest Palindromic Substring
11	Container With Most Water
15	3Sum
19	Remove Nth Node From End of List
20	Valid Parentheses
21	Merge Two Sorted Lists
22	Generate Parentheses
23	Merge k Sorted Lists
26	Remove Duplicates from Sorted Array
28	Find the Index of the First Occurrence in a String
33	Search in Rotated Sorted Array
34	Find First and Last Position of Element in Sorted Array
35	Search Insert Position
39	Combination Sum
45	Jump Game II
46	Permutations
47	Permutations II
49	Group Anagrams
50	Pow(x, n)
51	N-Queens
55	Jump Game
56	Merge Intervals
57	Insert Interval
70	Climbing Stairs

ID	Original problem statement
72	Edit Distance
76	Minimum Window Substring
78	Subsets
79	Word Search
84	Largest Rectangle in Histogram
98	Validate Binary Search Tree
104	Maximum Depth of Binary Tree
110	Balanced Binary Tree
127	Word Ladder
128	Longest Consecutive Sequence
131	Palindrome Partitioning
133	Clone Graph
136	Single Number
139	Word Break
141	Linked List Cycle
142	Linked List Cycle II
146	LRU Cache
155	Min Stack
167	Two Sum II - Input Array Is Sorted
191	Number of 1 Bits
198	House Robber
200	Number of Islands
201	Bitwise AND of Numbers Range
206	Reverse Linked List
207	Course Schedule
208	Implement Trie (Prefix Tree)
209	Minimum Size Subarray Sum
210	Course Schedule II
211	Design Add and Search Words Data Structure
212	Word Search II

ID	Original problem statement
215	Kth Largest Element in an Array
218	The Skyline Problem
232	Implement Queue using Stacks
236	Lowest Common Ancestor of a Binary Tree
238	Product of Array Except Self
239	Sliding Window Maximum
242	Valid Anagram
295	Find Median from Data Stream
300	Longest Increasing Subsequence
303	Range Sum Query - Immutable
307	Range Sum Query - Mutable
309	Best Time to Buy and Sell Stock with Cooldown
312	Burst Balloons
315	Count of Smaller Numbers After Self
322	Coin Change
327	Count of Range Sum
329	Longest Increasing Path in a Matrix
332	Reconstruct Itinerary
337	House Robber III
338	Counting Bits
347	Top K Frequent Elements
377	Combination Sum IV
380	Insert Delete GetRandom O(1)
416	Partition Equal Subset Sum
424	Longest Repeating Character Replacement
435	Non-overlapping Intervals
452	Minimum Number of Arrows to Burst Balloons
486	Predict the Winner
493	Reverse Pairs
494	Target Sum

ID	Original problem statement
496	Next Greater Element I
518	Coin Change II
525	Contiguous Array
543	Diameter of Binary Tree
547	Number of Provinces
560	Subarray Sum Equals K
643	Maximum Average Subarray I
684	Redundant Connection
703	Kth Largest Element in a Stream
704	Binary Search
718	Maximum Length of Repeated Subarray
721	Accounts Merge
739	Daily Temperatures
743	Network Delay Time
746	Min Cost Climbing Stairs
752	Open the Lock
787	Cheapest Flights Within K Stops
834	Sum of Distances in Tree
862	Shortest Subarray with Sum at Least K
864	Shortest Path to Get All Keys
875	Koko Eating Bananas
907	Sum of Subarray Minimums
974	Subarray Sums Divisible by K
981	Time Based Key-Value Store
994	Rotting Oranges
1011	Capacity To Ship Packages Within D Days
1109	Corporate Flight Bookings
1143	Longest Common Subsequence
1192	Critical Connections in a Network
1235	Maximum Profit in Job Scheduling

ID	Original problem statement
1368	Minimum Cost to Make at Least One Valid Path in a Grid
1392	Longest Happy Prefix
1483	Kth Ancestor of a Tree Node
1584	Min Cost to Connect All Points
1631	Path With Minimum Effort
1755	Closest Subsequence Sum
1851	Minimum Interval to Include Each Query
2035	Partition Array Into Two Arrays to Minimize Sum Difference
2376	Count Special Integers